

VARIOUS LEVELS OF TESTING

PHASES

NAME: APURWA ASHOK PARAB

ADMISSION NO.: HPGD/AP16/XXXX

SPECIALIZATION: IT PROJECTS MANAGEMENT

**PRIN. L. N. WELINGKAR INSTITUTE OF MANAGEMENT
DEVELOPMENT & RESEARCH**

March 2018

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the course of this PGDBA project. I am thankful for their aspiring guidance, invaluable constructive criticism and friendly advice during the project work. I am sincerely grateful to them for sharing their truthful and illuminating views on several issues related to the project.

I express my warm thanks to Mr. GertLittooij (Product Owner at NationaleNederlanden) and Mr. Ankit Pachori (Lead at Capgemini India Pvt Ltd) for their support and guidance.

Finally, off course, I am particularly grateful to my family for having believed in me and their constant support.

Thank you,
XXXX XXX XXXXX

UNDERTAKING BY CANDIDATE

I declare that project work entitled " Various Levels Of Testing Phases " .

is my own work conducted as part of my syllabus .

I further declare that project work presented has been prepared personally by me and it is not sourced from any outside agency. I understand that, any such malpractice will have very serious consequence and my admission to the program will be cancelled without any refund of fees.

I am also aware that, I may face legal action, if I follow such malpractice.



Signature of Candidate

Table Of Contents

SR. NO.	PARTICULARS	PAGE NO.
1	INTRODUCTION	5
2	SOFTWARE TESTING LIFE CYCLE	11
	2.1 Requirement Analysis	13

SAMPLE PROJECT

	2.2 Test Planning	14
	2.3 Test Case Development	15
	2.4 Environment Setup	16
	2.5 Test Execution	17
	2.6 Test Cycle Closure	18
3	LEVELS OF TESTING	19
	3.1 Functional Testing	19
	3.1.1 Unit Testing	20
	3.1.2 Component Testing	21
	3.1.3 Integration Testing	23
	3.1.4 System Testing	28
	3.1.5 Acceptance Testing	29
	3.2 Non-Functional Testing	33
	3.2.1 Performance Testing	33
	3.2.1.1 Load Testing	34
	3.2.1.2 Stress Testing	35
	3.2.1.3 Volume Testing	36
	3.2.2 Usability Testing	36
	3.2.3 Security Testing	38
	3.2.4 Portability Testing	39
	3.2.5 Configuration Testing	40
	3.2.6 Compatibility Testing	40
	3.2.7 Installability Testing	47
	3.2.8 Reliability Testing	48
	3.2.9 Recovery Testing	50
	3.2.10 Documentation Testing	51
	3.3 Automation Testing	52
4	Conclusion	63
5	Bibliography	64

INTRODUCTION

Before we move on to learning the various level of testing first let us learn about testing.



What is Software Testing?

Software testing is a process of executing a program or application with the intent of finding the **software bugs**. **Software testing** is an investigation conducted to provide stakeholders with information about the quality of the software product or service under test.

Software testing can also provide an objective, independent view of the software to allow the business to appreciate and understand the risks of software implementation. Test techniques include the process of executing a program or application with the intent of finding software bugs (errors or other defects), and verifying that the software product is fit for use.

Software testing involves the execution of a software component or system component to evaluate one or more properties of interest. In general, these properties indicate the extent to which the component or system under test.

- Meets the requirements that guided its design and development
- Responds correctly to all kinds of inputs,
- Performs its functions within an acceptable time,
- Is sufficiently usable,
- Can be installed and run in its intended environments, and
- Achieves the general result its stakeholder's desire.

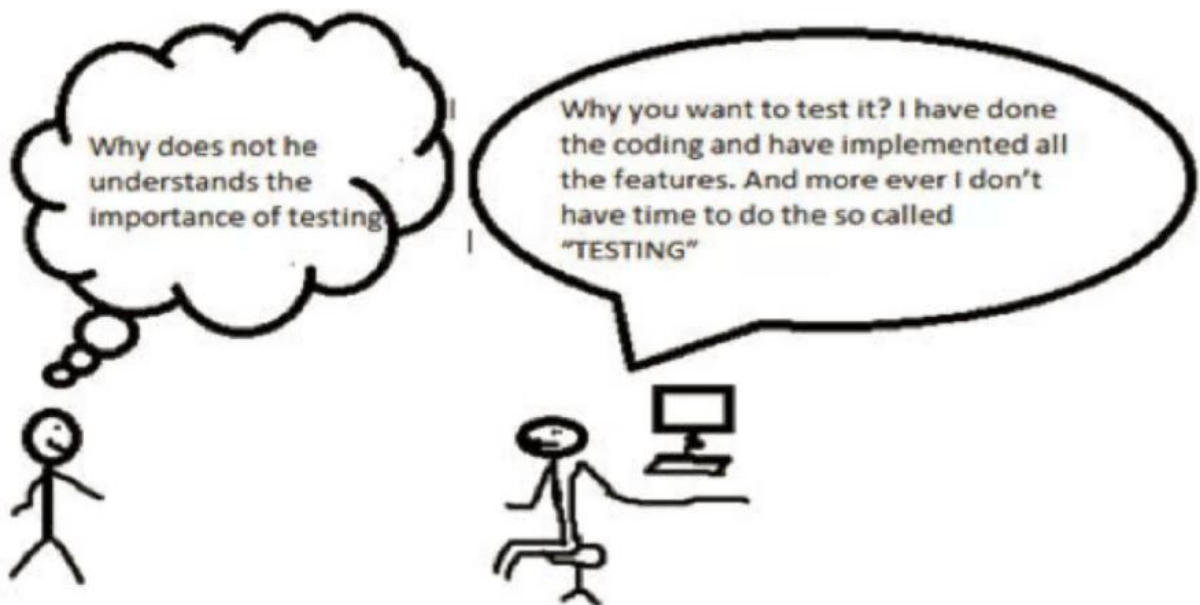
As the number of possible tests for even simple software components is practically infinite, all software testing uses some strategy to select tests that are feasible for the available time and resources. As a result, software testing typically (but not exclusively) attempts to execute a program or application with the intent of finding software bugs (errors or other defects). The job of testing is an iterative process as when one bug is fixed, it can illuminate other, deeper bugs, or can even create new ones.

Software testing can provide objective, independent information about the quality of software and risk of its failure to users or sponsors.

Software testing can be conducted as soon as executable software (even if partially complete) exists. The overall approach to software development often determines when and how testing is conducted. For example, in a phased process, most testing occurs after system requirements have been defined and then implemented in testable programs. In contrast, under an agile approach, requirements, programming, and testing are often done concurrently.



Why is software testing necessary?



Software Testing is necessary because we all make mistakes. Some of those mistakes are unimportant, but some of them are expensive or dangerous. We need to check everything and anything we produce because things can always go wrong – **humans make mistakes all the time.**

Since we assume that our work may have mistaken, hence we all need to check our own work. However some mistakes come from bad assumptions and blind spots, so we might make the same mistakes when we check our own work as we made when we did it. So we may not notice the flaws in what we have done.

Ideally, we should get someone else to check our work because another person is more likely to spot the flaws.

There are several reasons which clearly tells us as why Software Testing is important and what are the major things that we should consider while testing of any product or application.

Software testing is very important because of the following reasons:

1. Software testing is really required to point out the **defects** and errors that were made during the **development phases**.
2. It's essential since it makes sure of the Customer's reliability and their satisfaction in the application.
3. It is very important to ensure the Quality of the product. Quality product delivered to the customers helps in gaining their confidence. (Know more about **Software Quality**)
4. Testing is necessary in order to provide the facilities to the customers like the delivery of high quality product or software application which requires lower maintenance cost and hence results into more accurate, consistent and reliable results.
5. Testing is required for an effective performance of software application or product.
6. It's important to ensure that the application should not result into any **failures** because it can be very expensive in the future or in the later stages of the development.
7. It's required to stay in the business.



What are software testing objectives and purpose?

Software Testing has different goals and objectives. The major objectives of Software testing are as follows:

- **Finding defects** which may get created by the programmer while developing the software.

- Gaining confidence in and providing information about the level of **quality**.
- To prevent defects.
- To make sure that the end result meets the business and user requirements.
- To ensure that it satisfies the BRS that is Business Requirement Specification and SRS that is System Requirement Specifications.
- To gain the confidence of the customers by providing them a quality product.

Software testing helps in finalizing the software application or product against business and user requirements. It is very important to have good test coverage in order to test the software application completely and make it sure that it's performing well and as per the specifications.

While determining the **test coverage** the test cases should be designed well with maximum possibilities of finding the errors or bugs. The **test cases** should be very effective. This objective can be measured by the number of defects reported per test cases. Higher the number of the defects reported the more effective are the test cases.

Once the delivery is made to the end users or the customers they should be able to operate it without any complaints. In order to make this happen the tester should know as how the customers are going to use this product and accordingly they should write down the test scenarios and design the test cases. This will help a lot in fulfilling all the customer's requirements.

Software testing makes sure that the testing is being done properly and hence the system is ready for use. Good coverage means that the testing has been done to cover the various areas like functionality of the application, **compatibility** of the application with the OS, hardware and different types of browsers, **performance testing** to test the performance of the application and **load testing** to make sure that the system is reliable and should not crash or there should not be any blocking issues. It also determines that the application can be deployed easily to the machine and without any resistance. Hence the application is easy to install, learn and use.



What are the principles of testing?

1) Testing shows presence of defects: Testing can show the **defects** are present, but cannot prove that there are no defects. Even after testing the application or product thoroughly we cannot say that the product is 100% defect free. Testing always reduces the number of undiscovered defects remaining in the software but even if no defects are found, it is not a proof of correctness.

2) Exhaustive testing is impossible: Testing everything including all combinations of inputs and preconditions is not possible. So, instead of doing the exhaustive testing **we can use risks and priorities** to focus testing efforts. For example: In an application in one screen there are 15 input fields, each having 5 possible values, then to test all the valid combinations you would need 5^{15} tests. This is very unlikely that the project timescales would allow for this number of tests. So, accessing and managing risk is one of the most important activities and reason for testing in any project.

3) Early testing: In the **software development life cycle** testing activities should start as early as possible and should be focused on defined objectives.

4) Defect clustering: A small number of modules contains most of the defects discovered during **pre-release testing** or shows the most operational failures.

5) Pesticide paradox: If the same kinds of tests are repeated again and again, eventually the same set of test cases will no longer be able to find any new bugs. To overcome this “Pesticide Paradox”, it is really very important to review the test cases regularly and new and different tests need to be written to exercise different parts of the software or system to potentially find more defects.

6) Testing is context dependent: Testing is basically context dependent. Different kinds of sites are tested differently. For example, safety – critical software is tested differently from an e-commerce site.

7) Absence – of – errors fallacy: If the system built is unusable and does not fulfil the user's needs and expectations then finding and fixing defects does not help.



2. Software Testing Life Cycle

Software Testing Life Cycle (STLC) is defined as a sequence of activities conducted to perform Software Testing.

It consists of series of activities carried out methodologically to help certify your software product.

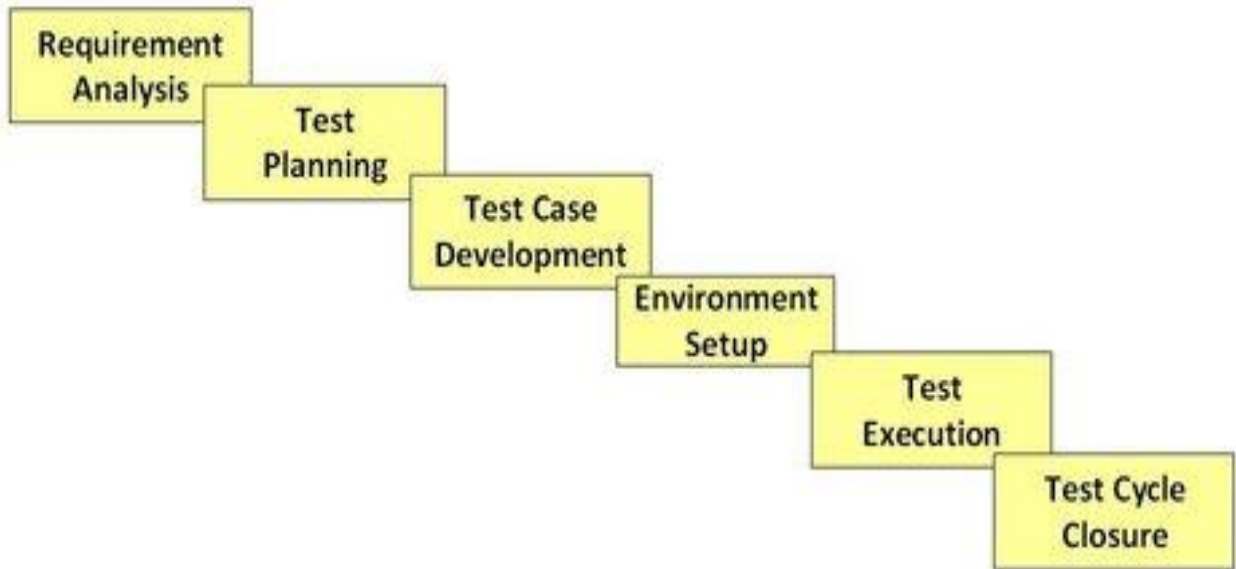


Diagram - Different stages in Software Test Life Cycle

2.1 Requirement Analysis



During this phase, test team studies the requirements from a testing point of view to identify the testable requirements.

The QA team may interact with various stakeholders (Client, Business Analyst, Technical Leads, and System Architects etc.) to understand the requirements in detail.

Requirements could be either Functional (defining what the software must do) or Non-Functional (defining system performance /security availability)

Automation feasibility for the given testing project is also done in this stage.

Activities

- Identify types of tests to be performed.
- Gather details about testing priorities and focus.
- Prepare Requirement Traceability Matrix (RTM).
- Identify test environment details where testing is supposed to be carried out.
- Automation feasibility analysis (if required).

Deliverables

- RTM
- Automation feasibility report. (if applicable)

2.2 Test Planning

Test planning is the second step of the testing process. In this phase typically Test Manager/Test Lead involves determining the effort and cost estimates for the entire project.

Preparation of Test Plan will be done based on the requirement analysis. Activities like resource planning, determining roles and responsibilities, tool selection (if automation), training requirement etc., carried out in this phase. The deliverables of this phase are Test Plan & Effort estimation documents.

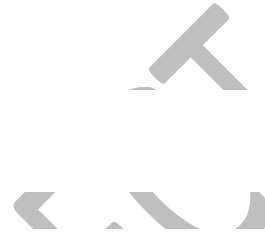


Activities

- Preparation of test plan/strategy document for various types of testing
- Test tool selection
- Test effort estimation
- Resource planning and determining roles and responsibilities.
- Training requirement

Deliverables

- Test plan / Strategy document.
- Effort estimation document.



2.3 Test Case Development

Test team starts with test cases development activity here in this phase. Test team prepares test cases, test scripts (if automation) and test data. Once the test cases are ready then these test cases are reviewed by peer members or team lead.

Also, test team prepares the Requirement Traceability Matrix (RTM). RTM traces the requirements to the test cases that are needed to verify whether the requirements are fulfilled. The deliverables of this phase are Test Cases, Test Scripts, Test Data, Requirements Traceability Matrix.



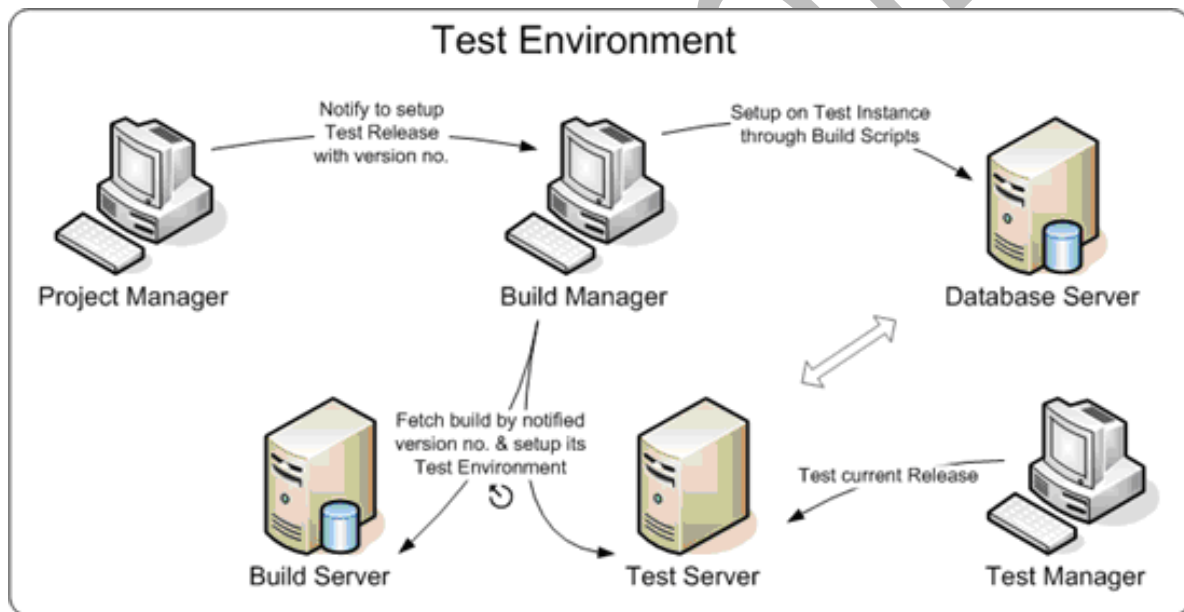
Activities

- Create test cases, automation scripts (if applicable)
- Review and baseline test cases and scripts
- Create test data (If Test Environment is available)

Deliverables

- Test cases/scripts
- Test data

2.4 Test Environment Setup



Test environment decides the software and hardware conditions under which a work product is tested. Test environment set-up is one of the critical aspects of testing process and **can be done in parallel with Test Case Development Stage**.

Test team may not be involved in this activity if the customer/development team provides the test environment in which case the test team is required to do a readiness check (smoke testing) of the given environment.

Activities

- Understand the required architecture, environment set-up and prepare hardware and software requirement list for the Test Environment.
- Setup test Environment and test data
- Perform smoke test on the build

Deliverables

- Environment ready with test data set up
- Smoke Test Results.

2.5 Test Execution



During this phase the testers will carry out the testing based on the test plans and the test cases prepared. Bugs will be reported back to the development team for correction and retesting will be performed.

Activities

- Execute tests as per plan
- Document test results, and log defects for failed cases
- Map defects to test cases in RTM
- Retest the Defect fixes
- Track the defects to closure

Deliverables

- Completed RTM with execution status
- Test cases updated with results
- Defect reports

2.6 Test Cycle Closure



Testing team will meet, discuss and analyze testing artifacts to identify strategies that must be implemented in future, taking lessons from the current test cycle. The idea is to remove the process bottlenecks for future test cycles and share best practices for any similar projects in future.

Activities

- Evaluate cycle completion criteria based on Time, Test coverage, Cost, Software, Critical Business Objectives and Quality.
- Prepare test metrics based on the above parameters.
- Document the learning out of the project.
- Prepare Test closure report.
- Qualitative and quantitative reporting of quality of the work product to the customer.
- Test result analysis to find out the defect distribution by type and severity.

Deliverables

- Test Closure report
- Test metrics

Now that we have learnt the importance of testing and the software testing life cycle, let's us move on to learn about the various levels of testing phases.

Various Levels of Testing Phases

There are different levels during the process of testing. Levels of testing include different methodologies that can be used while conducting software testing. The main levels of software testing are:

- Functional Testing
- Non-functional Testing

3.1 Functional Testing

This is a type of black-box testing that is based on the specifications of the software that is to be tested. The application is tested by providing input and then the results are examined that need to conform to the functionality it was intended for. Functional testing of a software is conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements.

There are five steps that are involved while testing an application for functionality.

Steps	Description
I	The determination of the functionality that the intended application is meant to perform.
II	The creation of test data based on the specifications of the application.
III	The output based on the test data and the specifications of the application.
IV	The writing of test scenarios and the execution of test cases.
V	The comparison of actual and expected results based on the executed test cases.

An effective testing practice will see the above steps applied to the testing policies of every organization and hence it will make sure that the organization maintains the strictest of standards when it comes to software quality.

Testing levels are basically to identify missing areas and prevent overlap and repetition between the development life cycle phases. In software development life cycle models there are defined phases like requirement gathering and analysis, design, coding or implementation, testing and deployment. Each phase goes through the testing. Hence there are various levels of testing. The various levels of testing are:



- 1) **Unit Test**
- 2) **Component Test**
- 3) **Integration Test**
 - a. **Component Integration Test**
 - b. **System Integration Test**
- 4) **System Test**
- 5) **Acceptance Test**
 - a. **Alpha Testing**
 - b. **Beta Testing**

Let us look through every level in detail.

3.1.1 UNIT TESTING



A **unit test** is the smallest testable part of an application like functions, classes, procedures, interfaces. Unit testing is a method by which individual units of source code are tested to determine if they are fit for use.

It is basically done by the developers to make sure that their code is working fine and meet the user specifications. They test their piece of code which they have written like classes, functions, interfaces and procedures.

The goal of unit testing is to segregate each part of the program and test that the individual parts are working correctly.

This means that for any function or procedure when a set of inputs are given then it should return the proper values. It should handle the failures gracefully during the course of execution when any invalid input is given.

A unit test provides a written contract that the piece of code must assure. Hence it has several benefits.

Advantages of Unit testing:

1. Issues are found at early stage. Since unit testing are carried out by developers where they test their individual code before the integration. Hence the issues can be found very early and can be resolved then and there without impacting the other piece of codes.
2. Unit testing helps in maintaining and changing the code. This is possible by making the codes less interdependent so that unit testing can be executed. Hence chances of impact of changes to any other code gets reduced.
3. Since the bugs are found early in unit testing hence it also helps in reducing the cost of bug fixes. Just imagine the cost of bug found during the later stages of development like during system testing or during acceptance testing.
4. Unit testing helps in simplifying the debugging process. If suppose a test fails then only latest changes made in code needs to be debugged.

3.1.2 COMPONENT TESTING

Component testing is a method where testing of each component in an application is done separately. Suppose, in an application there are 5 components. Testing of each 5 components separately and efficiently is called as component testing.

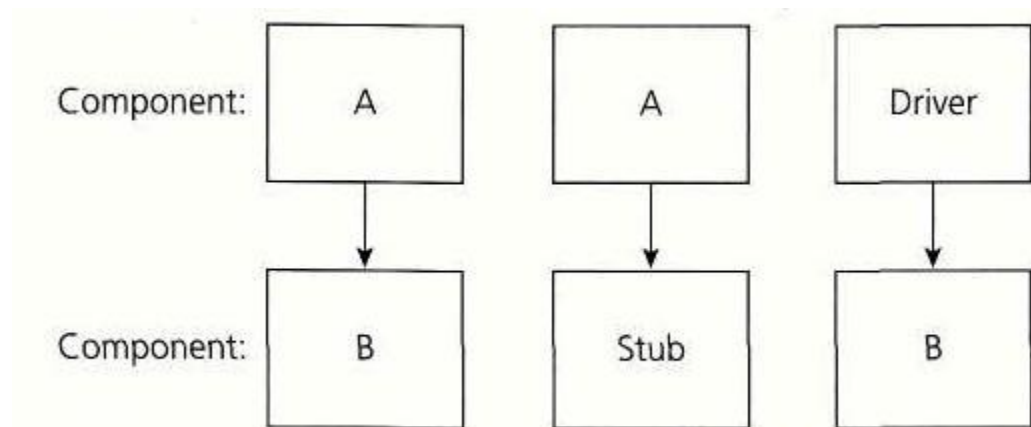
It is also called as module testing. The basic difference between the unit testing and component testing is in unit testing the developers test their piece of code but in component testing the whole component is tested. For example, in a student record application there are two modules one which will save the records of the students and other module is to upload the results of the students. Both the modules are developed separately and when they are tested one by one then we call this as a component or module testing.

- Component testing is done by the tester.
- Component testing may be done in isolation from rest of the system depending on the development life cycle model chosen for that particular application. In such case the missing software is replaced by **Stubs** and **Drivers** and simulate the interface between the software components in a simple manner.
- Let's take an example to understand it in a better way. Suppose there is an application consisting of three modules say, module A, module B and module C. The developer has

developed the module B and now wanted to test it. But in order to test the module B completely few of it's functionalities are dependent on module A and few on module C. But the module A and module C has not been developed yet. In that case to test the module B completely we can replace the module A and module C by stub and drivers as required.

- **Stub:** A stub is called from the software component to be tested. As shown in the diagram below 'Stub' is called by 'component A'.
- **Driver:** A driver calls the component to be tested. As shown in the diagram below 'component B' is called by the 'Driver'.

Below is the diagram of the component testing:



As discussed in the previous article of the 'Unit testing' it is done by the developers where they do the testing of the individual functionality or procedure. After unit testing is executed, component testing comes into the picture. Component testing is done by the testers.

Component testing plays a very important role in finding the bugs. Before we start with the integration testing it's always preferable to do the component testing in order to ensure that each component of an application is working effectively.

Integration testing is followed by the component testing.

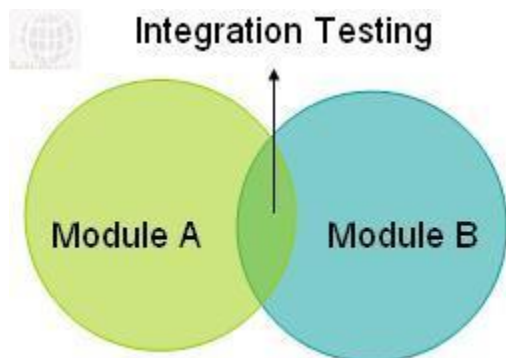
3.1.3 INTEGRATION TESTING

Integration testing is done when two modules are integrated, in order to test the behavior and functionality of both the modules after integration. Below are few types of integration testing:

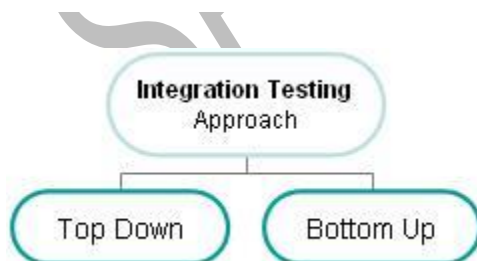
- Big bang integration testing
- Top down
- Bottom up
- Functional incremental

Integration testing tests integration or interfaces between components, interactions to different parts of the system such as an operating system, file system and hardware or interfaces between systems.

- Also after integrating two different **components** together we do the integration **testing**. As displayed in the image below when two different modules 'Module A' and 'Module B' are integrated then the integration testing is done.



- Integration testing is done by a specific integration tester or test team.
- Integration testing follows two approach known as 'Top Down' approach and 'Bottom Up' approach as shown in the image below:

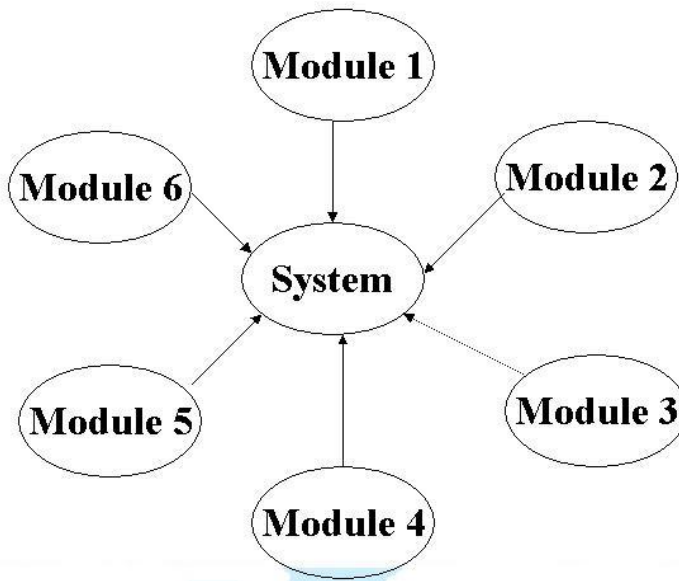


Below are the integration testing techniques:

1. Big Bang integration testing:

In Big Bang integration testing all components or modules are integrated simultaneously, after which everything is tested as a whole. As per the below image all the modules from 'Module 1' to 'Module 6' are integrated simultaneously then the testing is carried out.

Big Bang Integration Testing



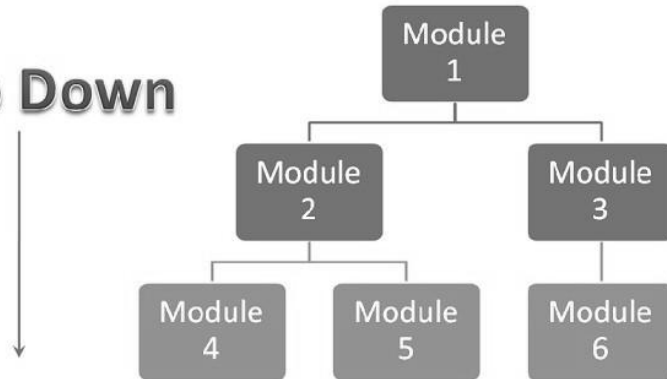
Advantage: Big Bang testing has the advantage that everything is finished before integration testing starts.

Disadvantage: The major disadvantage is that in general it is time consuming and difficult to trace the cause of failures because of this late integration.

2. **Top-down integration testing:** Testing takes place from top to bottom, following the control flow or architectural structure (e.g. starting from the GUI or main menu). Components or systems are substituted by stubs. Below is the diagram of 'Top down Approach':



Top Down



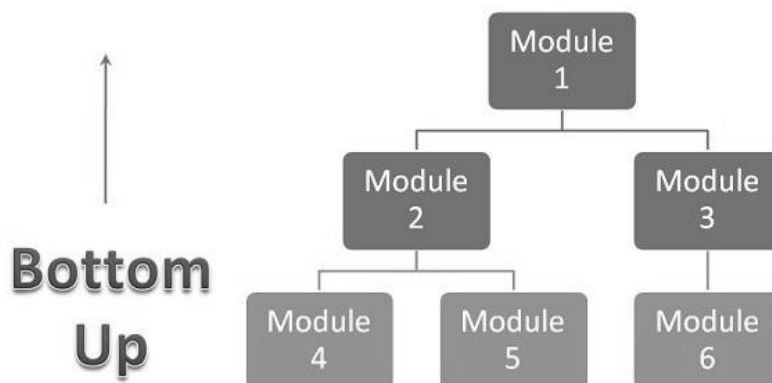
Advantages of Top-Down approach:

- The tested product is very consistent because the integration testing is basically performed in an environment that almost similar to that of reality
- Stubs can be written with lesser time because when compared to the drivers then Stubs are simpler to author.

Disadvantages of Top-Down approach:

- Basic functionality is tested at the end of cycle

3. **Bottom-up integration testing:** Testing takes place from the bottom of the control flow upwards. Components or systems are substituted by drivers. Below is the image of 'Bottom up approach':



Advantage of Bottom-Up approach:

- In this approach development and testing can be done together so that the product or application will be efficient and as per the customer specifications.

Disadvantages of Bottom-Up approach:

- We can catch the Key interface defects at the end of cycle
- It is required to create the test drivers for modules at all levels except the top control

Incremental testing:

- Another extreme is that all programmers are integrated one by one, and a test is carried out after each step.
- The incremental approach has the advantage that the defects are found early in a smaller assembly when it is relatively easy to detect the cause.
- A disadvantage is that it can be time-consuming since stubs and drivers have to be developed and used in the test.
- Within incremental integration testing a range of possibilities exist, partly depending on the system architecture.

Functional incremental: Integration and testing takes place on the basis of the functions and functionalities, as documented in the functional specification.

3.1.4 COMPONENT INTEGRATION TESTING

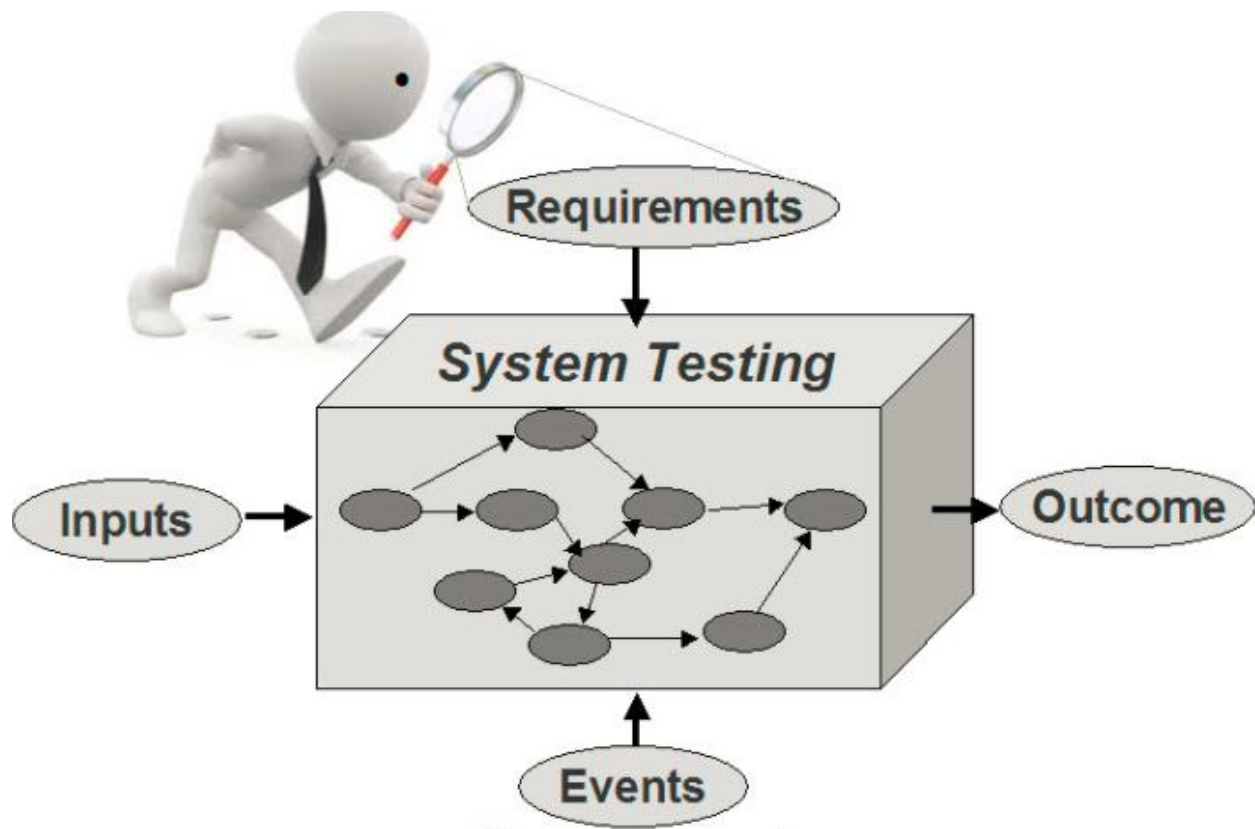
- It tests the interactions between software components and is done after component testing.
- The software components themselves may be specified at different times by different specification groups, yet the integration of all of the pieces must work together.
- It is important to cover negative cases as well because components might make assumption with respect to the data.

3.1.5 SYSTEM INTEGRATION TESTING

System integration testing (SIT) is a testing where testers basically test that in the same environment all the related systems should maintain data integrity and can operate in coordination with other systems.

- System integration testing (SIT) tests the interactions between different systems and may be done after **system testing**.
- It verifies the proper execution of software components and proper interfacing between components within the solution.
- The objective of SIT Testing is to validate that all software module dependencies are **functionally correct** and that data integrity is maintained between separate modules for the entire solution.
- As testing for dependencies between different components is a primary function of SIT Testing, this area is often most subject to **Regression Testing**.

3.1.6 SYSTEM TESTING



- In **system testing** the behavior of whole system/product is tested as defined by the scope of the development project or product.
- It may include **tests based on risks** and/or **requirement specifications**, business process, use cases, or other high level descriptions of system behavior, interactions with the operating systems, and system resources.
- System testing is most often the final test to verify that the system to be delivered meets the specification and its purpose.
- System testing is carried out by specialists testers or **independent testers**.
- System testing should investigate both **functional** and **non-functional requirements** of the testing.

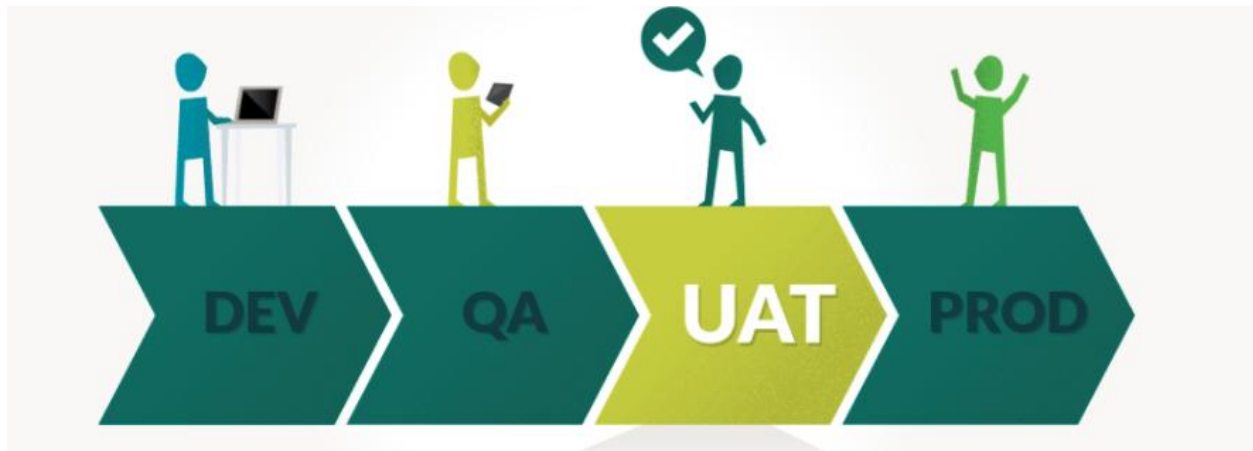
3.1.6.1 Regression Testing

Whenever a change in a software application is made, it is quite possible that other areas within the application have been affected by this change. Regression testing is performed to verify that a fixed bug hasn't resulted in another functionality or business rule violation. The intent of regression testing is to ensure that a change, such as a bug fix should not result in another fault being uncovered in the application.

Regression testing is important because of the following reasons:

- Minimize the gaps in testing when an application with changes made has to be tested.
- Testing the new changes to verify that the changes made did not affect any other area of the application.
- Mitigates risks when regression testing is performed on the application.
- Test coverage is increased without compromising timelines.
- Increase speed to market the product.

3.1.7 ACCEPTANCE TESTING



After the system test has corrected all or most defects, the system will be delivered to the user or customer for **Acceptance Testing** or **User Acceptance Testing (UAT)**.

- Acceptance testing is basically done by the user or customer although other stakeholders may be involved as well.
- The goal of acceptance **testing** is to establish confidence in the system.
- Acceptance testing is most often focused on a validation type testing.
- Acceptance testing may occur at more than just a single level, for example:
 - A **Commercial Off the shelf (COTS)** software product may be acceptance tested when it is installed or integrated.
 - **Acceptance testing of the usability of the component** may be done during component testing.
 - **Acceptance testing of a new functional enhancement** may come before **system testing**.

The **types of acceptance testing** are:

- The **User Acceptance test**: focuses mainly on the functionality thereby validating the fitness-for-use of the system by the business user. The user acceptance test is performed by the users and application managers.
- The **Operational Acceptance test**: also known as Production acceptance test validates whether the system meets the requirements for operation. In most of the organization the operational acceptance test is performed by the system administration before the system is released. The operational acceptance test

may include testing of backup/restore, disaster recovery, maintenance tasks and periodic check of security vulnerabilities.

- **Contract Acceptance testing:** It is performed against the contract's acceptance criteria for producing custom developed software. Acceptance should be formally defined when the contract is agreed.
- **Compliance acceptance testing:** It is also known as regulation acceptance testing is performed against the regulations which must be adhered to, such as governmental, legal or safety regulations.

ALPHA TESTING

Alpha testing is one of the most common **software testing strategy** used in software development. It's specially used by product development organizations.

- This **test takes place at the developer's site**. Developers observe the users and note problems.
- Alpha testing is testing of an application when development is about to complete. Minor design changes can still be made as a result of alpha testing.
- Alpha testing is typically performed by a group that is independent of the design team, but still within the company, e.g. in-house software test engineers, or software QA engineers.
- Alpha testing is final testing before the software is released to the general public. It has two phases:
 - In the **first phase** of alpha testing, the software is tested by in-house developers. They use either debugger software, or hardware-assisted debuggers. The goal is to catch bugs quickly.
 - In the **second phase** of alpha testing, the software is handed over to the software QA staff, for additional testing in an environment that is similar to the intended use.
- Alpha testing is simulated or actual operational testing by potential users/customers or an independent test team at the developers' site. Alpha testing is often employed for off-the-shelf software as a form of internal acceptance testing, before the software goes to beta testing.

BETA TESTING

Beta Testing is also known as field testing. It takes place at **customer's site**. It sends the system/software to users who install it and use it under real-world working conditions.

- A beta test is the second phase of software testing in which a sampling of the intended audience tries the product out. (Beta is the second letter of the Greek alphabet.) Originally, the term alpha testing meant the first phase of testing in a software development process. The first phase includes unit testing, component testing, and system testing. Beta testing can be considered “pre-release testing.”
- The goal of beta testing is to place your application in the hands of real users outside of your own engineering team to discover any flaws or issues from the user's perspective that you would not want to have in your final, released version of the application. Example: Microsoft and many other organizations release beta versions of their products to be tested by users.

Closed beta versions are released to a select group of individuals for a user test and are invitation only, while Open betas are from a larger group to the general public and anyone interested. The testers report any bugs that they find, and sometimes suggest additional features they think should be available in the final version.

Advantages of beta testing

- You have the opportunity to get your application into the hands of users prior to releasing it to the general public.
- Users can install, test your application, and send feedback to you during this beta testing period.
- Your beta testers can discover issues with your application that you may have not noticed, such as confusing application flow, and even crashes.
- Using the feedback you get from these users, you can fix problems before it is released to the general public.
- The more issues you fix that solve real user problems, the higher the quality of your application when you release it to the general public.

- Having a higher-quality application when you release to the general public will increase customer satisfaction.
- These users, who are early adopters of your application, will generate excitement about your application.

3.2 Non-Functional Testing

This section is based upon testing an application from its non-functional attributes. Non-functional testing involves testing a software from the requirements which are nonfunctional in nature but important such as performance, security, user interface, etc.

Some of the important and commonly used non-functional testing types are discussed below.

3.2.1 Performance Testing



It is mostly used to identify any bottlenecks or performance issues rather than finding bugs in a software. There are different causes that contribute in lowering the performance of a software:

- Network delay

- Client-side processing
- Database transaction processing
- Load balancing between servers
- Data rendering

Performance testing is considered as one of the important and mandatory testing type in terms of the following aspects:

- Speed (i.e. Response Time, data rendering and accessing)
- Capacity
- Stability
- Scalability

Performance testing can be either qualitative or quantitative and can be divided into different sub-types such as **Load testing**, **Volume testing** and **Stress testing**.

1. Load Testing



It is a process of testing the behavior of a software by applying maximum load in terms of software accessing and manipulating large input data. It can be done at both normal and peak load conditions. This type of testing identifies the maximum capacity of software and its behavior at peak time.

Most of the time, load testing is performed with the help of automated tools such as Load Runner, AppLoader, IBM Rational Performance Tester, Apache JMeter, Silk Performer, Visual Studio Load Test, etc.

Virtual users (VUsers) are defined in the automated testing tool and the script is executed to verify the load testing for the software. The number of users can be increased or decreased concurrently or incrementally based upon the requirements.

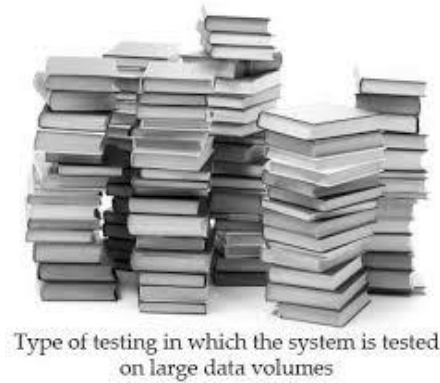
2. Stress Testing



Stress testing puts the program under heavy load or stress. According to Glenford Myers [1], "...a heavy stress is a peak volume of data or activity, encountered **over a short period of time**." Stress testing is very easily confused with volume testing. However, the key difference is the element of time involved in stress testing.

Borrowing from a good example from Glenford Myers "**The Art of Software Testing, Second Edition**", while a volume test would test if a typist could cope with typing up a large report, a stress tests would test if the typist could type at a rate of 55 words per minute. Stress testing is usually applicable to programs with varying loads. A good candidate for stress testing is an online application that interacts with a database. A typical stress test would try to determine the maximum number of concurrent users the application could handle and also explore the systems reaction when the number of concurrent users exceeds the maximum.

3. Volume Testing



In volume testing, the program under test is subjected to huge volumes of data to see how it copes. For example, a graphics application would be fed a ridiculously large bitmap file to edit or a compiler would be fed with source files containing outrageous amounts of code. The aim of volume testing is to show that the system cannot handle the amount of data it has specified in its objectives.

3.2.2 Usability Testing



Usability testing is a black-box technique and is used to identify any error(s) and improvements in the software by observing the users through their usage and operation.

According to Nielsen, usability can be defined in terms of five factors, i.e. efficiency of use, learn-ability, memory-ability, errors/safety, and satisfaction. According to him, the usability of a product will be good, and the system is usable if it possesses the above factors.

Usability testing is concerned with determining how easily a human would interface with the system or utilize the system. When designing usability test cases, it is important to study the end users the program is targeted towards. Important factors to consider would be the age of the audience, the educational background and possibly the inclusion of accessibility features for the disabled should be considered. It is also important to ensure that the program outputs meaningful, non-offensive messages.

Nigel Bevan and Macleod considered that usability is the quality requirement that can be measured as the outcome of interactions with a computer system. This requirement can be fulfilled, and the end-user will be satisfied if the intended goals are achieved effectively with the use of proper resources.

Molich in 2000 stated that a user-friendly system should fulfill the following five goals, i.e., easy to Learn, easy to remember, efficient to use, satisfactory to use, and easy to understand.

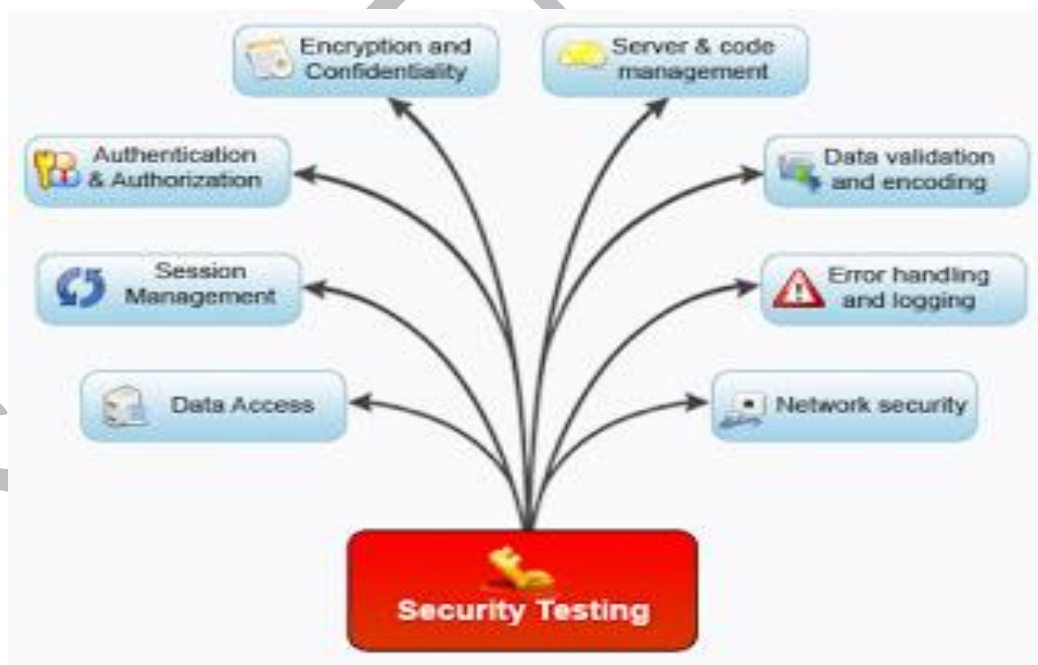
In addition to the different definitions of usability, there are some standards and quality models and methods that define usability in the form of attributes and sub-attributes such as ISO-9126, ISO-9241-11, ISO-13407, and IEEE std.610.12, etc.

➤ UI vs Usability Testing

UI testing involves testing the Graphical User Interface of the Software. UI testing ensures that the GUI functions according to the requirements and tested in terms of color, alignment, size, and other properties.

On the other hand, usability testing ensures a good and user-friendly GUI that can be easily handled. UI testing can be considered as a sub-part of usability testing.

3.2.3 Security Testing



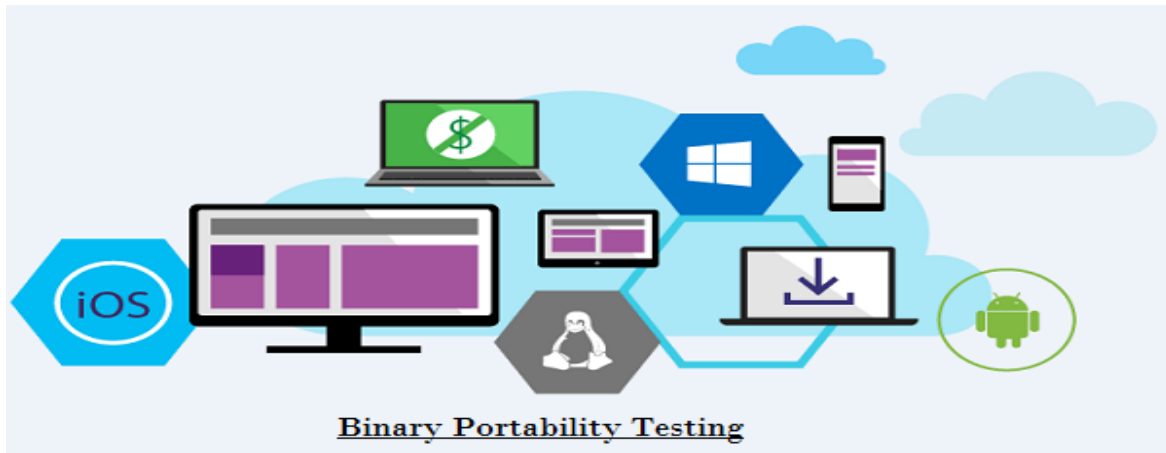
Security tests attempts to break the program's security. Test cases would normally be designed to attempt to access resources that should not be accessible at all or without the required

privileges. An example would be an attempt to corrupt the data in a database system. Internet based application are a good candidate for security testing due to the continuous growth in the number of e-commerce applications.

Security testing involves testing a software in order to identify any flaws and gaps from security and vulnerability point of view. Listed below are the main aspects that security testing should ensure:

- Confidentiality
- Integrity
- Authentication
- Availability
- Authorization
- Non-repudiation
- Software is secure against known and unknown vulnerabilities
- Software data is secure
- Software is according to all security regulations
- Input checking and validation
- SQL insertion attacks
- Injection flaws
- Session management issues
- Cross-site scripting attacks
- Directory traversal attacks

3.2.4 Portability Testing



Portability testing includes testing a software with the aim to ensure its reusability and that it can be moved from another software as well. Following are the strategies that can be used for portability testing:

- Transferring an installed software from one computer to another.
- Building executable (.exe) to run the software on different platforms.

Portability testing can be considered as one of the sub-parts of system testing, as this testing type includes overall testing of a software with respect to its usage over different environments. Computer hardware, operating systems, and browsers are the major focus of portability testing. Some of the pre-conditions for portability testing are as follows:

- Software should be designed and coded, keeping in mind the portability requirements.
- Unit testing has been performed on the associated components.
- Integration testing has been performed.
- Test environment has been established.

3.2.5 Configuration Testing

Configuration Testing



Application functions well with different types of hardware technologies, drivers, operating systems.

Many software systems are now built to operate under multiple operating systems and different hardware configurations. Web applications are a special case as there are many different web browsers and a given web browser would operate differently on different operating systems. Configuration testing therefore determines how a system performs under different software and hardware configurations.

3.2.6 Compatibility Testing

Many programs are built to replace old systems. As a result, these programs usually have compatibility requirement with the old system. A good example would be the replacement of an application that utilizes a database management system as its back end data store. It is clear that the new application must be compatible with the old database. Hence compatibility test aims to show that the system compatibility objectives were not met.



What is Software Compatibility Testing?

Computer has become an essential part of our lives. Several software applications have been developed to help tech people in their careers, work, shop, and in many other actions.

Online purchasing is very common now days. While selling the product or software, online seller must keep in mind that the product he is selling should be bug free otherwise seller might lose business and reputation while buyer of the software may waste his or her money in buying defective software.

To endure the competitive market, it is a necessity that the software or applications you provide to buyers are worth it the amount they are paying. To deliver the good quality product it is very important the application or software goes through different stages of the development in terms of quality, compatibility, reliability and delivery.



What is Software compatibility?



Compatibility is the ability of living and work together without any discrepancy. Compatible software applications also work on same setups. For example, if Google.com site is compatible, then it should open in all browsers and operating system.



What is Software Compatibility Testing?



Compatibility is a non- functional testing to ensure customer satisfaction. It is to determine whether your software application or product is proficient enough to run in different browsers, database, hardware, operating system, mobile devices and networks. Application could also impact due to different versions, resolution, internet speed and configuration etc. Hence, it's important to test the application in all possible manners to reduce failures and overcome from embarrassments of bug's leakage. As a Non- functional test, Compatibility testing is to endorse that the application runs properly in different browsers, versions, OS and networks successfully.

Compatibility test should always perform on real environment instead of virtual environment. Test the compatibility of application with different browsers and operating systems to guarantee 100% coverage.

Types of Software compatibility testing:

- Browser compatibility testing
- Hardware
- Networks
- Mobile Devices
- Operating System
- Versions

Browser compatibility testing

It is very popular in compatibility testing. It is to check the compatibility of the software application on different browsers like Chrome, Firefox, Internet Explorer, Safari, and Opera etc.

Hardware

it is to check the application/ software compatibility with the different hardware configurations.

Network

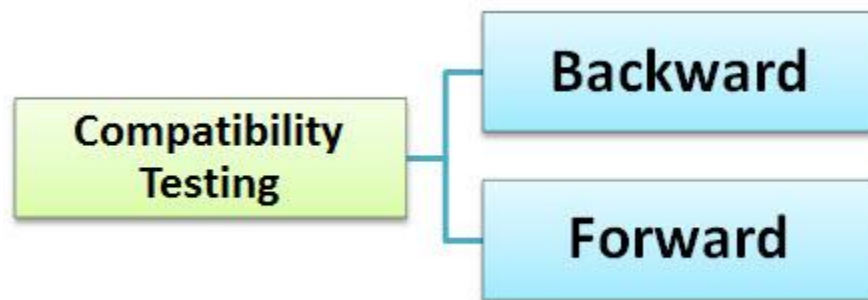
it is to check the application in different network like 3G, WIFI etc.

Mobile Devices

it is to check if the application is compatible with the mobile devices and their platforms like android, iOS, windows etc.

Operating Systems

it is to check if application is compatible with different Operating Systems like Windows, Linux, and Mac etc.



- **Versions**

It is important to test the software applications in different versions of the software. There are two different types of version inspection.

- **Backward Compatibility Testing**– Testing of the application or software in old or previous versions. It is also known as downward compatible.
- **Forward Compatibility Testing**-Testing of the application or software in new or upcoming versions. It is also known as forward compatible



Why we perform Compatibility testing?

Compatibility testing is to check the application working in same way for all platforms. Usually dev team and testing team test the application in the single platform. But the once application released in the production, customer may test our product in different platform and they may find bugs in the application which is not worthy in terms of quality. To reduce such issues and not to upset your customers it is important to test the application in all platforms.

When should perform Compatibility testing?

When build gets stable enough to test then we should perform compatibility testing.

Common Compatibility testing defects

- Changes in UI (look and feel)
- Change in font size
- Alignment related issues
- Change in CSS style and color
- Scroll bar related issues
- Content or label overlapping
- Broken tables or Frames



Pick what to test as compatibility testing



Make a note of most important testing parameter for your application where you feel application can behave weirdly. Decide versions of browsers, operating systems and devices where you would like to test your application. Best practice is to analyze the requirement and cross check with the client or customer for browser matrix. Let customer decide which all browsers, OS and version they would like us to test the application.

With the help of [Google Analytic](#) or alternative type of statistical analysis system setup on your application can give you the clear statistics of the widely used browser with their version and operating system.

Select pages to test

Filter out the main URLs, pages of your application. Selection of the pages totally depends on your application. You would require considering the majorly used modules as a part of compatibility testing. If your application consists of certain template format, it's fine if you consider that only as a part of compatibility testing.



How to perform Compatibility testing?

1. **Test the application in same browsers but in different versions.** For e.g. to test the compatibility of site ebay.com. Download different versions of Firefox and install them one by one and test the eBay site. eBay site should behave equally same in each version.

2. **Test the application in different browsers but in different versions.** For e.g. testing of site ebay.com in different available browsers like Firefox, Safari, Chrome, Internet Explorer and Opera etc.

Tools for compatibility testing



BrowserStack - Browser Compatibility Testing:

This tool helps us to check your application in different browsers.

Secure Platform - Hardware Compatibility tool

This tool includes necessary drivers for a specific hardware platform and it provides information on a tool to check for CD burning process with CD burning tools.

3.2.7 Installability Testing



Installability testing is a testing in which one will perform testing on the application in order to conform whether the guide line provided in installation documents suitable for installing the application into environment properly or not.

Installation testing may also be considered as an activity-based approach to how to test something. For example, install the software in the various ways and on the various types of systems that it can be installed. Check which files are added or changed on disk. Does the installed software work? What happens when you uninstall?

This testing is typically performed in Operational acceptance testing, by a software testing engineer in conjunction with the configuration manager. Implementation testing is usually defined as testing which places a compiled version of code into the testing or pre-production environment, from which it may or may not progress into production.

In distributed systems, particularly where software is to be released into an already live target environment (such as an operational website) installation (or software deployment as it is sometimes called) can involve database schema changes as well as the installation of new software. Deployment plans in such circumstances may include back-out procedures whose use is intended to roll the target environment back if the deployment is unsuccessful. Ideally, the deployment plan itself should be tested in an environment that is a replica of the live

environment. A factor that can increase the organizational requirements of such an exercise is the need to synchronize the data in the test deployment environment with that in the live environment with minimum disruption to live operation.

3.2.8 Reliability Testing



Reliability Testing is about exercising an application so that failures are discovered and removed before the system is deployed. The purpose of reliability testing is to determine product reliability, and to determine whether the software meets the customer's reliability requirements.

- According to ANSI, Software Reliability is defined as: the probability of failure-free software operation for a specified period of time in a specified environment. Software Reliability is not a direct function of time. Electronic and mechanical parts may become “old” and wear out with time and usage, but software will not rust or wear-out during its life cycle. Software will not change over time unless intentionally changed or upgraded.
- Reliability refers to the consistency of a measure. A test is considered reliable if we get the same result repeatedly. Software Reliability is also an important factor affecting system reliability.
- Reliability testing will tend to uncover earlier those failures that are most likely in actual operation, thus directing efforts at fixing the most important faults.

- Reliability testing may be performed at several levels. Complex systems may be tested at component, circuit board, unit, assembly, subsystem and system levels.

Software reliability is a key part in software quality. The study of software reliability can be categorized into three parts:

1. Modeling
2. Measurement
3. Improvement

1. Modeling: Software reliability modeling has matured to the point that meaningful results can be obtained by applying suitable models to the problem. There are many models exist, but no single model can capture a necessary amount of the software characteristics. Assumptions and abstractions must be made to simplify the problem. There is no single model that is universal to all the situations.

2. Measurement: Software reliability measurement is naive. Measurement is far from commonplace in software, as in other engineering field. “How good is the software, quantitatively?” As simple as the question is, there is still no good answer. Software reliability cannot be directly measured, so other related factors are measured to estimate software reliability and compare it among products. Development process, faults and failures found are all factors related to software reliability.

3. Improvement: Software reliability improvement is hard. The difficulty of the problem stems from insufficient understanding of software reliability and in general, the characteristics of software. Until now there is no good way to conquer the complexity problem of software. Complete testing of a moderately complex software module is infeasible. Defect-free software product cannot be assured. Realistic constraints of time and budget severely limits the effort put into software reliability improvement.

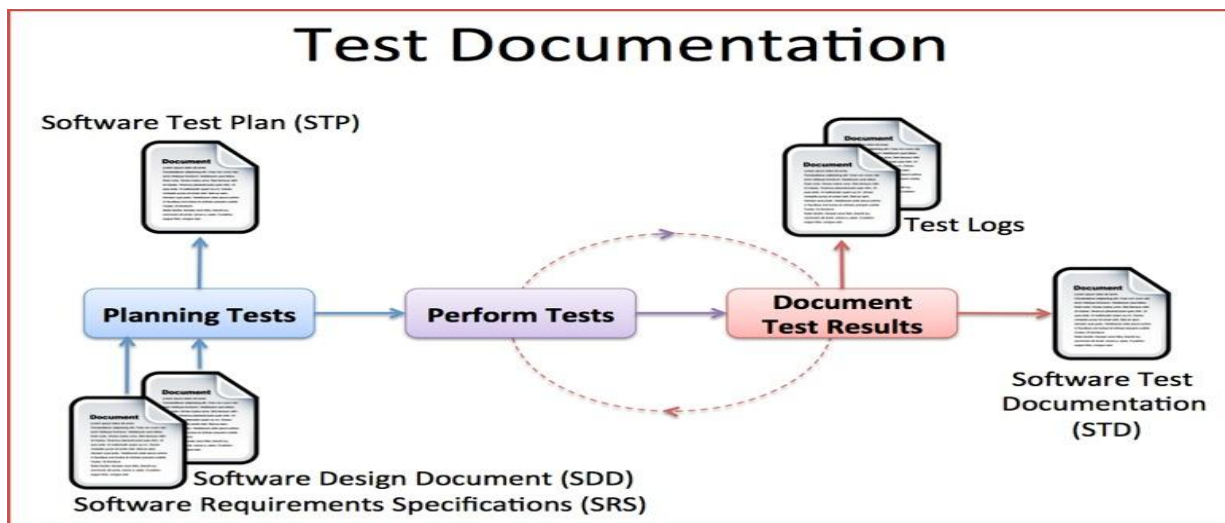
3.2.9 Recovery Testing



1. It is a type of **non-functional testing**.
2. Recovery testing is done in order to check how fast and better the application can recover after it has gone through any type of crash or hardware failure etc.
3. Recovery testing is the forced **failure** of the software in a variety of ways to verify that recovery is properly performed.
4. For example: When an application is receiving data from a network, unplug the connecting cable. After some time, plug the cable back in and analyze the application's ability to continue receiving data from the point at which the network connection was broken.
5. **Example:** Restart the system while a browser has a definite number of sessions and check whether the browser can able to recover all of them or not.

It involves reverting to a point where the integrity of the system was known and then reprocessing transactions up to the point of failure.

3.2.10 Documentation Testing



- It is a type of **non-functional testing**.
- Any written or pictorial information describing, defining, specifying, reporting, or certifying activities, requirements, procedures, or results'. Documentation is as important to a product's success as the product itself. If the documentation is poor, non-existent, or wrong, it reflects on the quality of the product and the vendor.
- As per the IEEE Documentation describing plans for, or results of, the testing of a system or component, Types include test case specification, test incident report, test log, test plan, test procedure, test report. Hence the testing of all the above-mentioned documents is known as documentation testing.
- This is one of the most cost-effective approaches to testing. If the documentation is not right: there will be major and costly problems. The documentation can be tested in a number of different ways to many different degrees of complexity. These range from running the documents through a spelling and grammar checking device, to manually reviewing the documentation to remove any ambiguity or inconsistency.
- Documentation testing can start at the very beginning of the software process and hence save large amounts of money, since the earlier a **defect** is found the less it will cost to be fixed.

3.3 Automation Testing



What is Automation Testing?

Manual Testing is performed by a human sitting in front of a computer carefully executing the test steps. Automation Testing means using an automation tool to execute your test case suite.

The automation software can also enter test data into the System under Test, compare expected and actual results and generate detailed test reports.

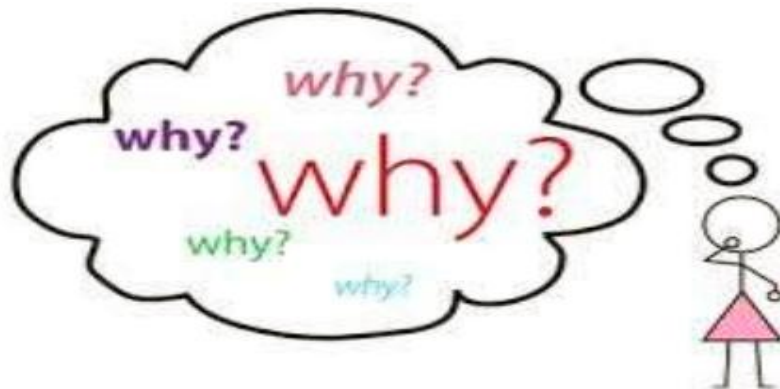
Test Automation demands considerable investments of money and resources. Successive development cycles will require execution of same test suite repeatedly. Using a test automation tool, it's possible to record this test suite and re-play it as required.

Once the test suite is automated, no human intervention is required. This improved ROI of Test Automation. Goal of Automation is to reduce number of test cases to be run manually and not eliminate Manual Testing all together.



Why Automated Testing?

Why Automation Testing is important ?



Automated software testing is important due to following reasons:

- Manual Testing of all work flows, all fields, all negative scenarios is time and cost consuming
- It is difficult to test for multi lingual sites manually
- Automation does not require Human intervention. You can run automated test unattended (overnight)
- Automation increases speed of test execution
- Automation helps increase Test Coverage
- Manual Testing can become boring and hence error prone.



Which Test Cases to Automate?

Test cases to be automated can be selected using the following criterion to increase the automation ROI

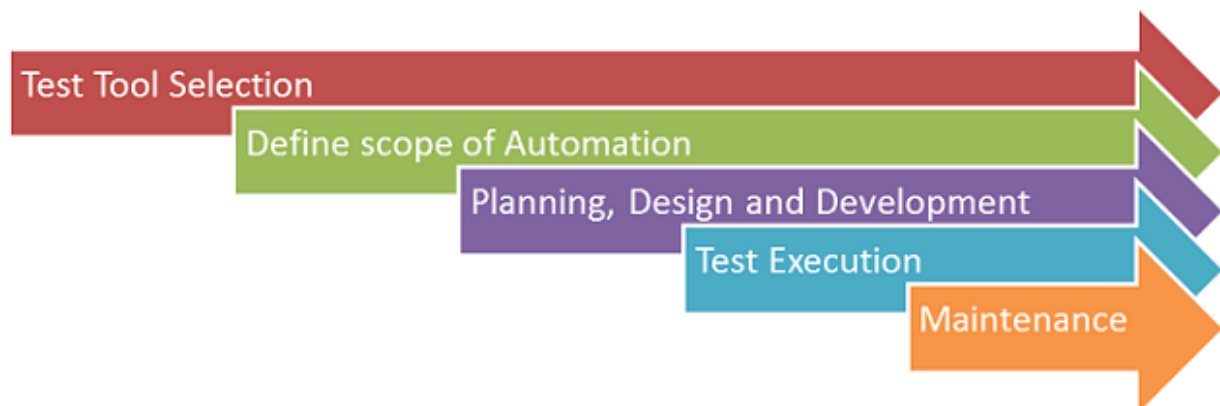
- High Risk - Business Critical test cases
- Test cases that are executed repeatedly
- Test Cases that are very tedious or difficult to perform manually
- Test Cases which are time consuming

The following category of test cases are not suitable for automation:

- Test Cases that are newly designed and not executed manually at least once
- Test Cases for which the requirements are changing frequently
- Test cases which are executed on ad-hoc basis.

Automated Testing Process:

Following steps are followed in an Automation Process-



Test tool selection

Test Tool selection largely depends on the technology the Application under Test is built on. For instance, QTP does not support Informatics. So QTP cannot be used for testing Informatics applications. **It's a good idea to conduct Proof of Concept of Tool on AUT.**

Define the scope of Automation

Scope of automation is the area of your Application under Test which will be automated. Following points help determine scope:

- Feature that are important for the business
- Scenarios which have **large amount of data**
- **Common functionalities** across applications
- Technical feasibility
- Extent to which business components are reused
- **Complexity** of test cases
- Ability to use the same test cases for cross browser testing.

Planning, Design and Development

During this phase you create Automation strategy & plan, which contains following details-

- Automation tools selected
- Framework design and its features
- In-Scope and Out-of-scope items of automation
- Automation test bed preparation
- Schedule and Timeline of scripting and execution
- Deliverables of Automation Testing

Test Execution

Automation Scripts are executed during this phase. The scripts need input test data before they are set to run. Once executed they provide detailed test reports.

Execution can be performed using the automation tool directly or through the Test Management tool which will invoke the automation tool.

Example: Quality center is the Test Management tool which in turn it will invoke QTP for execution of automation scripts. Scripts can be executed in a single machine or a group of machines. The execution can be done during night, to save time.

- **Maintenance**

As new functionalities are added to the System under Test with successive cycles, Automation Scripts need to be added, reviewed and maintained for each release cycle. **Maintenance becomes necessary to improve effectiveness of Automation Scripts.**

Framework in Automation

A framework is set of automation guidelines which help in

- Maintaining consistency of Testing
- Improves test structuring
- Minimum usage of code
- Less Maintenance of code
- Improve re-usability
- Non-Technical testers can be involved in code
- Training period of using the tool can be reduced
- Involves Data wherever appropriate

There are four types of framework used in automation software testing:

1. Data Driven Automation Framework
2. Keyword Driven Automation Framework

3. Modular Automation Framework
4. Hybrid Automation Framework

Automation Tool Best Practices

To get maximum ROI of automation, observe the following

- Scope of Automation needs to be determined in detail before the start of the project. This sets expectations from Automation right.
- Select the right automation tool: A tool must not be selected based on its popularity, but it's fit to the automation requirements.
- Choose appropriate framework
- Scripting Standards- Standards have to be followed while writing the scripts for Automation. Some of them are-
 - Create uniform scripts, comments and indentation of the code
 - Adequate Exception handling - How error is handled on system failure or unexpected behavior of the application.
 - User defined messages should be coded or standardized for Error Logging for testers to understand.
- Measure metrics- Success of automation cannot be determined by comparing the manual effort with the automation effort but by also capturing the following metrics.
 - Percent of defects found
 - Time required for automation testing for each and every release cycle
 - Minimal Time taken for release
 - Customer satisfaction Index
 - Productivity improvement

The above guidelines if observed can greatly help in making your automation successful.

Benefits of Automation Testing

Following are benefits of automated testing:

- 70% faster than the manual testing
- Wider test coverage of application features
- Reliable in results
- Ensure Consistency
- Saves Time and Cost
- Improves accuracy
- Human Intervention is not required while execution
- Increases Efficiency
- Better speed in executing tests
- Re-usable test scripts
- Test Frequently and thoroughly
- More cycle of execution can be achieved through automation
- Early time to market

Different types of software testing that can be automated

- Smoke Testing
- Unit Testing
- Integration Testing
- Functional Testing
- Keyword Testing
- Regression Testing
- Data Driven Testing
- Black Box Testing



How to Choose an Automation Tool?

Selecting the right tool can be a tricky task. Following criterion will help you select the best tool for your requirement-

- Environment Support

- Ease of use
- Testing of Database
- Object identification
- Image Testing
- Error Recovery Testing
- Object Mapping
- Scripting Language Used
- Support for various types of test - including functional, test management, mobile
- Support for multiple testing frameworks
- Easy to debug the automation software scripts
- Ability to recognize objects in any environment
- Extensive test reports and results
- Minimize training cost of selected tools

Tool selection is one of biggest challenges to be tackled before going for automation. First, Identify the requirements, explore various tools and its capabilities, set the expectation from the tool and go for a Proof of Concept.

Automation Testing Tools

There are tons of Functional and Regression Testing Tools available in market. Here are 5 best tools certified by our experts

1. Selenium

It is a software testing tool used for Regression Testing. It is an open source testing tool that provides playback and recording facility for Regression Testing. The Selenium IDE only supports Mozilla Firefox web browser.

- It provides the provision to export recorded script in other languages like Java, Ruby, RSpec, Python, C#, etc
- It can be used with frameworks like Junit and TestNG
- It can execute multiple tests at a time
- Autocomplete for Selenium commands that are common

- Walkthrough tests
- Identifies the element using id, name, X-path, etc.
- Store tests as Ruby Script, HTML, and any other format
- It provides an option to assert the title for every page
- It supports selenium user-extensions.js file
- It allows to insert comments in the middle of the script for better understanding and debugging

2. HP UFT (QTP)

QTP is widely used for functional and regression testing, it addresses every major software application and environment. To simplify test creation and maintenance, it uses the concept of keyword driven testing. It allows the tester to build test cases directly from the application.

- It is easier to use for non-technical person to adapt to and create working test cases
- It fixes defects faster by thoroughly documenting and replicating defects for developer
- Collapse test creation and test documentation at a single site
- Parameterization is easy than WinRunner
- QTP supports .NET development environment
- It has better object identification mechanism
- It can enhance existing QTP scripts without "Application Under Test" being available, by using the ActiveScreen

3. Rational Functional Tester

It is an Object-Oriented Automated Functional Testing tool that can perform automated functional, regression, data-driven testing and GUI testing. The main features of this tool are

- It supports a wide range of protocols and applications like Java, HTML, NET, Windows, SAP, Visual basic, etc.
- It can record and replay the actions on demand

- It integrates well with source control management tools such as Rational Clear Case and Rational Team Concert integration
- It allows developers to create keyword associated script so that it can be re-use
- Eclipse Java Developer Toolkit editor facilitates the team to code test scripts in Java with Eclipse
- It supports custom controls through proxy SDK (Java/.Net)
- It supports version control to enable parallel development of test scripts and concurrent usage by geographically distributed team.

4. SilkTest

Silk Test is designed for doing functional and regression testing. For e-business application, silk test is the leading functional testing product. It is a product of Segue Software takeover by Borland in 2006. It is an object-oriented language just like C++. It uses the concept of object, classes, and inheritance. Its main feature includes

- It consists of all the source script files
- It converts the script commands into GUI commands. On the same machine, commands can be run on a remote or host machine
- To identify the movement of mouse along with keystrokes, Silktest can be executed. It can avail both playback and record method or descriptive programming methods to get the dialogs
- It identifies all controls and windows of the application under test as objects and determine all the attributes and properties of each window

5. TestComplete

Testcomplete is a powerful and easy-to-use functional test automation tool from SmartBear. It can automate tests across desktop, mobile, and web applications.

TestComplete supports many scripting languages like VBScript, Python, and JavaScript as well as various testing techniques such as keyword-driven testing, data-driven testing, and regression testing, and distributed testing.

With the tool's record-and-replay feature and the robust object identification engine, you can also create complex automated test scripts without writing a single line of code.

6. Cucumber

Cucumber is an open-source Behavior Driven Development (BDD) tool. Cucumber supports languages like Ruby, Java, Scala, Groovy, etc.

Drafting test scripts is a joint effort of testers, developer, and customers. Cucumber supports only the web environment. Test code is written in simple English called Gherkin. Cucumber code can be executed on different frameworks like Selenium, Ruby, etc

The focus of cucumber is the end-user experience.

Conclusion

The cost of maintaining software is much more than the cost of producing the software. In order to reduce this maintenance cost, more time and resources is being invested in extensively testing the software before deployment.

We have seen different levels of testing i.e. Unit Test, Integration Test, System Test and Acceptance Test.

Unit testing is done at the lowest level of software components to ensure the implementation fit the functional specification. It is a method by which individual units of source code are tested to determine if they are fit for use

Integration testing is done when two modules are integrated, in order to test the behavior and functionality of both the modules after integration. Integration testing tests integration or interfaces between components, interactions to different parts of the system such as an operating system, file system and hardware or interfaces between systems.

System testing is done to determine if the functionality of the system as a whole is as was specified in the user requirements specification. It may include tests based on risks and/or requirement specifications, business process, use cases, or other high level descriptions of system behavior, interactions with the operating systems, and system resources

Acceptance testing verifies that the end user(s) is (or is not) satisfied with the system. We have also seen regression testing which occurs throughout the software maintenance phase.

Finally, it is important to test at different levels because sources of errors in the software can then be easily tracked and fixed. It also provides a working subsystem which can be compiled into libraries for later use in other projects or for sale as fully tested compiled library. The more defects you find during the initial phases the less cost it is required to fix them.

Bibliography



1. A Guide to the Project Management Body of Knowledge (PMBOK Guide) Fifth Edition
2. A Guide to the Project Management Body of Knowledge (PMBOK Guide) Fourth Edition
3. Glenford J. Myers. The Art of Software Testing, Second Edition 2004, John Wiley and Sons, Inc. Chapters 5 & 6.
4. www.wikipedia.com
5. Rodney Parkin. Software Unit Testing, IV & V Australia, The independent software testing specialists, 1997
6. <http://istqbexamcertification.com/what-are-software-testing-levels/>
7. <http://softwaretestingfundamentals.com/software-testing-life-cycle/>

SAM